

# Test Case Driven Analysis of Configuration Knowledge Bases

Paper #7944

## Abstract

Configuration knowledge bases are used to represent variability properties of products and services. These knowledge bases can become quite large and complex and – as a result – are in the need of a testing and diagnosis support. If some test cases become inconsistent with the configuration knowledge base, this means that some constraints in the knowledge base do not represent expected properties of the underlying product domain. In this paper, we present an approach that supports test case driven direct diagnosis of configuration knowledge bases. This diagnosis approach helps knowledge engineers identify minimal sets of faulty constraints in a knowledge base which can be regarded as an explanation of the observed faulty behavior. The results of a performance evaluation show significant improvements in terms of runtime performance compared to existing knowledge base testing and debugging approaches.

## 1 Introduction

Knowledge-based configurators [Bachant and McDermott, 1984; Stumptner, 1997; Junker, 2006; McGuinness, 2007] support users in deciding about which components should be included in a specific product instance. Configuration knowledge bases are a formal representation of variability properties of complex products and services. Example application areas of knowledge-based configuration are telecommunication systems [Fleischanderl *et al.*, 1998], the automotive domain [Sinz *et al.*, 2003], and railway interlocking systems [Falkner and Schreiner, 2014]. Progressing from rule-based knowledge representations [Bachant and McDermott, 1984], model-based representations such as constraint satisfaction problems (CSPs) [Junker, 2006], answer set programming (ASP) [Erdem *et al.*, 2016], Boolean satisfiability problems (SAT) [Sinz *et al.*, 2003], and description logics (DL) [McGuinness, 2007] have been developed, which allow for a clear separation of domain and problem solving knowledge.

Due to the increasing size and complexity of configuration knowledge bases, their engineering and maintenance becomes an error-prone process triggered a.o. by a cognitive

overload of knowledge engineers, non-existing product domain knowledge, and outdated constraints in a knowledge base [Bakker *et al.*, 1993; Felfernig *et al.*, 2004; Shchekotykhin *et al.*, 2012]. As a consequence, finding faulty constraints (rules) responsible for the faulty behavior of a knowledge base becomes a challenging task for engineers engaged in the development, testing, and debugging of configuration knowledge bases [Fleischanderl, 2002]. In this paper, we introduce a diagnosis approach that helps to identify minimal sets of faulty constraints that are responsible for the unintended behavior of a knowledge base. For demonstration purposes, we exemplify the application of this diagnosis approach in the context of constraint-based configuration, however, it can as well be applied with other types of model-based knowledge representations such as ASP, SAT, and DL.

Identifying the sources of an inconsistency can be regarded as an explanation task [Gupta *et al.*, 2021] with the goal to indicate those constraints that need to be adapted in order to be able to identify a solution. Related scenarios are: (1) the identification of minimal subsets of erroneous constraints in an inconsistent knowledge base [Bakker *et al.*, 1993; Schlobach *et al.*, 2007], (2) the identification of minimal subsets of user preferences inducing an inconsistency with the underlying knowledge base [Felfernig *et al.*, 2012], and (3) in the context of knowledge base testing and debugging, the identification of minimal subsets of constraints in a knowledge base responsible for an inconsistency [Felfernig *et al.*, 2004] which is *induced by a set of test cases* specifying (un)intended behavior.

There are two basic diagnosis approaches: (1) diagnosis determination in a 2-phase process starting with conflict detection [Junker, 2004] followed by conflict resolution (diagnosis) using, for example, hitting set directed acyclic graphs (HSDAGs) [Reiter, 1987; DeKleer *et al.*, 1992; Greiner *et al.*, 1992]. Following a breadth-first search regime results in minimal cardinality diagnoses (hitting sets). Based on the related seminal work of [Reiter, 1987; DeKleer *et al.*, 1992], a large body of research contributions has been developed including alternative diagnosis approaches such as constraint-based [Feldman *et al.*, 2010; Metodi *et al.*, 2012], compilation-based [Siddiqi and Huang, 2007], and duality-aware approaches [Stern *et al.*, 2012]. (2) direct diagnosis [Felfernig *et al.*, 2012; Nica and Wotawa, 2012; Cbe and Wassermann, 2015; Marques-Silva *et al.*, 2015] is

based on the idea of determining diagnoses directly, i.e., without the need of predetermining conflicts.

Direct diagnosis can be implemented with task-specific knowledge encodings. Such encodings have already been implemented in constraint-based settings [Nica and Wotawa, 2012], SAT solving [Metodi *et al.*, 2014; Marques-Silva *et al.*, 2015], and description logics [C  be and Wassermann, 2015]. Apart from such encodings, direct diagnosis can be implemented using divide-and-conquer algorithms that efficiently split-up the diagnosis search space for filtering out diagnosis-irrelevant constraints [Felfernig *et al.*, 2012]. In this paper, we focus on scenarios where inconsistencies in a knowledge base are induced by test cases which specify (in constraint form) intended and unintended configurations. Related work on diagnosis-based knowledge base testing and debugging is restricted to 2-phase hitting set based approaches [Felfernig *et al.*, 2004] which potentially create performance issues [Nica *et al.*, 2013].

In order to increase the efficiency of diagnosis search in knowledge base testing and debugging scenarios, we introduce an approach to the determination of subset-minimal constraints which are identified on the basis of the concepts of direct diagnosis [Felfernig *et al.*, 2012]. In this context, we adapt and extend direct diagnosis algorithms [Felfernig *et al.*, 2012] to be applicable in knowledge base testing and debugging scenarios. We show in detail how such direct diagnosis algorithms have to be adapted and extended in order to be able to take into account positive and negative test cases specifying the intended behavior of a knowledge base. Furthermore, we show how to explicitly define tradeoffs between diagnosis efficiency and minimality thus making direct diagnosis more amenable for specific performance requirements. We want to emphasize that alternative solutions in terms of direct encodings [Nica and Wotawa, 2012; Metodi *et al.*, 2014; Marques-Silva *et al.*, 2015; C  be and Wassermann, 2015] that take into account test suites do not exist up to now but are within the scope of our future research.

The major contributions of this paper are the following. (1) using configuration as an example domain, we introduce an algorithm (MSSDIRECT) that supports test case driven direct diagnosis of knowledge bases. This algorithm makes direct diagnosis applicable to knowledge base development and maintenance scenarios. (2) based on the results of a performance evaluation, we show that MSSDIRECT clearly outperforms existing knowledge base testing and debugging approaches [Felfernig *et al.*, 2004; Shchekotykhin *et al.*, 2012]. (3) with this, we help to make diagnosis more applicable in industrial knowledge base testing and debugging which is regarded as highly relevant [Fleischanderl, 2002].

The remainder of this paper is organized as follows. In Section 2, we introduce a definition of a configuration task and a corresponding configuration and provide a working example that is used for demonstration purposes. In Section 3, we introduce the basic concept of test case driven knowledge base diagnosis. Thereafter, we present our approach to the test-driven direct diagnosis of configuration knowledge bases (see Section 4). In Section 5, we present the results of a performance analysis and discuss corresponding algorithmic complexity properties. We conclude the paper with a discus-

sion of open issues for future research (Section 6).

## 2 Example Configuration Scenario

In the following, we represent a configuration task as a constraint satisfaction problem (CSP) [Junker, 2006; Rossi *et al.*, 2006]. Such a task consists of a set of variables  $V$  with corresponding finite domain definitions and a set of constraints  $CON$  representing restrictions regarding the combination of individual variable values. In addition to constraints representing product domain specific restrictions, user requirements are interpreted as an additional constraint set (see Definition 1 [Felfernig *et al.*, 2012]).

**Definition 1** (Configuration Task and Configuration Knowledge Base). A configuration task  $(V, CON)$  is defined by a set of variables  $V = \{v_1..v_n\}$  with corresponding domain definitions  $dom(v_\alpha)$  representing the set of values part of the domain of  $v_\alpha$ . Furthermore,  $CON = C_{KB} \cup C_{REQ}$  where  $C_{KB} = \{c_1..c_u\}$  is a set of constraints representing domain specific restrictions and  $C_{REQ} = \{c_{u+1}..c_q\}$  is a set of constraints representing user requirements.<sup>1</sup> Finally,  $(V, C_{KB})$  is denoted as configuration knowledge base.

A simple example of a configuration task is the following where *type* represents the car type, *fuel* represents the average fuel consumption per 100 kilometers, a *skibag* supports a convenient ski stowage inside a car, *4-wheel* is the actuation type (4-wheel supported or not supported), and *pdv* is the park distance control feature – see Example 1. The possible configurations are restricted by  $C_{KB} = \{c_1, c_2, c_3, c_4, c_5\}$  whereas  $C_{REQ}$  is assumed to be  $\{c_6, c_7\}$ .

**Example 1** (Configuration Task).

- $V = \{\text{type, fuel, skibag, 4-wheel, pdv}\}$
- $dom(\text{type}) = \{\text{city, limo, combi, xdrive}\},$   
 $dom(\text{fuel}) = \{4l, 6l, 10l\}, dom(\text{skibag}) = \{\text{yes, no}\},$   
 $dom(4\text{-wheel}) = \{\text{yes, no}\}, dom(\text{pdv}) = \{\text{yes, no}\}$
- $C_{KB} = \{c_1: 4\text{-wheel} = \text{yes} \rightarrow \text{type} = \text{xdrive}, c_2: \text{skibag} = \text{yes} \rightarrow \text{type} \neq \text{city}, c_3: \text{fuel} = 4l \rightarrow \text{type} = \text{city}, c_4: \text{fuel} = 6l \rightarrow \text{type} \neq \text{xdrive}, c_5: \text{type} = \text{city} \rightarrow \text{fuel} \neq 10l\}$
- $C_{REQ} = \{c_6: 4\text{-wheel} = \text{yes}, c_7: \text{skibag} = \text{yes}\}$

On the basis of the definition of such a configuration task, a constraint solver (configurator) is now able to determine a corresponding solution (given that  $C_{KB} \cup C_{REQ}$  is consistent). A definition of a configuration (solution to a configuration task) is the following (see Definition 2).

**Definition 2** (Configuration). A configuration for a configuration task  $(V, CON)$  is a variable assignment  $A = \{v_1 = a_1..v_n = a_n\}$  where  $a_\alpha \in dom(v_\alpha)$ .

A configuration is *consistent* if the assignments in  $A$  are consistent with the constraints in  $CON$ . A configuration is *complete* if all  $v_i \in V$  are instantiated. Finally, a configuration is *valid* if it is consistent and complete.

A valid configuration for the configuration task shown in Example 1 is the following.

<sup>1</sup> $c_\alpha$  are constraint identifiers.

| ID          | test case (constraint)            | consistent with $C_{KB}$ |
|-------------|-----------------------------------|--------------------------|
| $t_{\pi 1}$ | $skibag = yes \wedge type = city$ | ×                        |
| $t_{\pi 2}$ | $fuel = 6l$                       | ✓                        |
| $t_{\pi 3}$ | $4-wheel = yes \wedge fuel = 6l$  | ×                        |
| $t_{\nu 1}$ | $type = city \wedge fuel = 10l$   | ×                        |
| $t_{\nu 2}$ | $type = city \wedge fuel = 6l$    | ✓                        |

Table 1: Example positive test cases  $T_{\pi} = \{t_{\pi 1}..t_{\pi 3}\}$  and negative test cases  $T_{\nu} = \{t_{\nu 1}, t_{\nu 2}\}$ .

**Example 2** (Valid Configuration).  $A = \{type = xdrive, fuel = 10l, skibag = yes, 4-wheel = yes, pdc = yes\}$ .

Especially in complex configuration knowledge bases, faulty constraints can lead to situations where a configurator shows an unexpected behavior, i.e., does not determine the expected solutions. For example, if the constraint  $c_5 : type = city \rightarrow fuel \neq 10l$  would have been defined as constraint  $c'_5 : type = city \rightarrow fuel = 10l$ , cars of type *city* are assumed to have an average *fuel* consumption of 10l. In the following, we show how to identify a minimal set of faulty constraints in a knowledge base (more precisely,  $C_{KB}$ ) that are responsible for a faulty behavior. These constraints need to be deleted or adapted to restore the consistency of the configuration knowledge base with regard to a predefined set of test cases (also denoted as examples) specifying the intended behavior of the configuration knowledge base. This means, our goal is to automatically identify sets of faulty constraints in order to make corresponding knowledge base development tasks less time-consuming for knowledge engineers.

### 3 Knowledge Base Diagnosis with Hitting Sets

Knowledge base testing follows the idea of regression testing where each of a given set of test cases is analyzed with regard to consistency with the constraints in  $C_{KB}$ . In the following, we differentiate between positive test cases  $T_{\pi}$  and negative test cases  $T_{\nu}$ . We use positive test cases to specify the intended behavior of a knowledge base and negative test cases to specify the unintended behavior of a knowledge base. An example of such test cases in the context of our working example is given in Table 1. We want to emphasize that test cases are not restricted to basic variable value assignments but can also be represented in a more general constraint form.

Test cases can be generated from different sources. For example, positive test cases can be derived from existing configurations, i.e., configurations determined in previous configuration sessions. Furthermore, positive as well as negative test cases can be manually defined by domain experts. Finally, test cases can be generated on the basis of a set of predefined well-formedness rules. A simple example of such a well-formedness rule is *each variable domain value has to be included in at least one consistent configuration*, i.e., there does not exist a domain value that cannot be integrated into at least one consistent configuration. Table 1 includes a simple set of test cases used in our working example.

For each individual positive test case  $t_{\pi}$  there should exist at least one solution for  $\{t_{\pi}\} \cup C_{KB}$ . Negative test cases  $t_{\nu}$  have to be inconsistent with the given knowledge base, i.e.,  $inconsistent(\{t_{\nu}\} \cup C_{KB})$ . For the set of negative test

cases, we expect  $\forall t_{\nu} \in T_{\nu} : inconsistent(C_{KB} \cup \{t_{\nu}\})$ . Furthermore, positive test cases are expected to be consistent with  $C_{KB}$ , i.e.,  $\forall t_{\pi} \in T_{\pi} : consistent(C_{KB} \cup \{t_{\pi}\})$ . In our approach, we also expect that positive and negative test cases are consistent with each other, i.e.,  $\forall t_{\pi} \in T_{\pi} : consistent(\{t_{\pi}\} \cup \overline{T_{\nu}})$  where  $\overline{T_{\nu}} = \bigwedge_{t_{\nu} \in T_{\nu}} \neg t_{\nu}$ .

In our example configuration knowledge base, *4-wheel* drive cars are not supported with engines consuming 6l of fuel. However, in our list of test cases (see Table 1), the positive test case  $t_{\pi 3}$  requires the existence of a configuration that includes both, *4-wheel = yes* and *fuel = 6l* (a new car type not taken into account might have this property). In this scenario, there exists at least one test case that expects configuration properties that cannot be fulfilled by the underlying domain-specific constraints in  $C_{KB}$ .

Reasons for such faulty configuration knowledge bases can range from non-existing product domain knowledge, modeling errors, i.e., a faulty translation of product domain constraints into the corresponding formal representation, or outdated constraints in the configuration knowledge base. Table 1 provides a full overview of our example test cases where  $t_{\pi 1}$ ,  $t_{\pi 3}$  should be accepted but are inconsistent with  $C_{KB}$ , and  $t_{\nu 2}$  should be rejected but is consistent with  $C_{KB}$ . The other test cases behave as expected.

A minimal diagnosis  $\Delta$  includes exactly those constraints responsible for the unexpected behavior of a configuration knowledge base. In our knowledge engineering context, constraints that have been identified as elements of  $\Delta$ , have to be deleted or adapted to restore the consistency between knowledge base and defined test cases. Definition 3 provides a formal definition of such a knowledge base diagnosis  $\Delta$ .

**Definition 3** (Knowledge Base Diagnosis). Given a configuration knowledge base  $(V, C_{KB})$  with associated positive test cases  $T_{\pi} = \{t_{\pi 1}..t_{\pi k}\}$  and negative test cases  $T_{\nu} = \{t_{\nu 1}..t_{\nu l}\}$ . A *knowledge base diagnosis*  $\Delta = \{c_1..c_r\} \subseteq C_{KB}$  has the property:  $\forall t_{\pi} \in T_{\pi} : consistent(C_{KB} - \Delta \cup \{t_{\pi}\} \cup \overline{T_{\nu}})$ .  $\Delta$  is minimal iff  $\nexists \Delta' \subset \Delta : knowledge\ base\ diagnosis(\Delta')$  (subset minimality).

Figure 1 provides an example of how such diagnoses can be determined on the basis of the concepts of model-based diagnosis [Reiter, 1987; Bakker *et al.*, 1993; Felfernig *et al.*, 2004]. The conjunction of the negated negative test cases ( $\overline{T_{\nu}} = \{\neg t_{\nu 1} \wedge \neg t_{\nu 2}\}$ ) can be regarded as background knowledge  $B$  which is assumed to be correct. If some of the negative test cases are (unexpectedly) consistent with  $C_{KB}$ , these test cases can be presented to the knowledge engineer as a direct indicator for needed refinements.

Let us assume that a conflict detection algorithm (e.g., QUICKXPLAIN [Junker, 2004; Rodler, 2022]) determines the first minimal conflict set for  $C_{KB}$  given a set of positive test cases  $T_{\pi}$ :  $CS_1 = \{c_2\}$  (induced by  $t_{\pi 1}$ ). In this context (see Figure 1), a *conflict set* (conflict)  $CS \subseteq C_{KB}$  is a set of constraints that satisfies the property  $\exists t_{\pi} \in T_{\pi} : inconsistent(t_{\pi} \cup CS \cup \overline{T_{\nu}})$ , i.e., at least one test case in  $T_{\pi}$  is inconsistent with  $CS \cup \overline{T_{\nu}}$ .  $CS$  is minimal iff  $\nexists CS' \subset CS : conflict(CS')$ .

After resolving  $CS_1$  by deleting  $c_2$  from  $C_{KB}$ , one further conflict can be identified in  $C_{KB} - \{c_2\}$  which is  $CS_2 =$

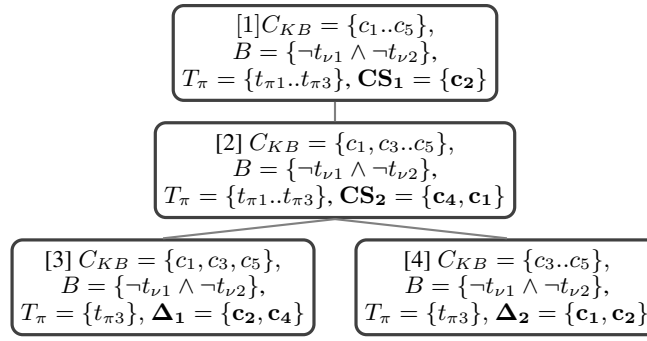


Figure 1: Hitting set directed acyclic graph (HSDAG) for  $C_{KB} = \{c_1..c_5\}$ ,  $T_\pi = \{t_{\pi 1}..t_{\pi 3}\}$ ,  $T_\nu = \{t_{\nu 1}, t_{\nu 2}\}$ . The resulting diagnoses (hitting sets) are  $\Delta_1 = \{c_2, c_4\}$  and  $\Delta_2 = \{c_1, c_2\}$ .

$\{c_4, c_1\}$  (induced by  $t_{\pi 3}$ ). Now, there are two options of resolving the second conflict: (1) when deleting  $c_4$ , the resulting diagnosis is  $\Delta_1 = \{c_2, c_4\}$ , (2) when deleting  $c_1$ ,  $\Delta_2 = \{c_1, c_2\}$  can be identified as second diagnosis. No further diagnosis can be identified since *consistent*( $C_{KB} - \Delta_1$ ) and *consistent*( $C_{KB} - \Delta_2$ ).

There exist two diagnoses indicating possible adaptations to restore the consistency between the constraints in  $C_{KB}$  and the given test cases (see Figure 1): either delete/adapt the constraints in  $\Delta_1$  or do this with the constraints of  $\Delta_2$ . For example,  $\Delta_2$  suggests to take a look at  $c_1$  (reconsider the relationship between the 4-wheel property and xdrive type cars) and  $c_2$  (reconsider the relationship between cars including skibags and non city type cars).

This hitting set based approach to knowledge base diagnosis ([Felfernig *et al.*, 2004]) is in the need of a conflict detection algorithm (such as QUICKXPLAIN [Junker, 2004]) that provides the basis for conflict resolution. In the following, we introduce a diagnosis approach which allows to omit the intermediate step of conflict detection and thus supports a more efficient diagnosis determination (one diagnosis at a time).

## 4 Test Case Driven Direct Diagnosis

The idea of direct diagnosis [Felfernig *et al.*, 2012] is to determine a (minimal) diagnosis without the need of conflict detection and a follow-up derivation of hitting sets (diagnoses) [Reiter, 1987]. In KBDIAG (see Algorithm 1), we assume  $C_{KB} = C \cup B$ , i.e., we differentiate between a consideration set  $C$  and a set of background constraints  $B$ . With this, we are able to focus the search for faulty knowledge base constraints: if we assume that the constraints in  $B$  are correct in any case, i.e., reflect the product domain knowledge, the diagnosis algorithm can focus diagnosis search on the consideration set  $C$ . The set  $\overline{T}_\nu$  represents a conjunction of all negative test cases in negated form.

Positive and negative test cases must not interfere with each other:  $\forall t_\pi \in T_\pi : \text{consistent}(\{t_\pi\} \cup \overline{T}_\nu)$ , i.e., each positive test case has to be consistent with  $\overline{T}_\nu$  (TESTC( $\overline{T}_\nu, T_\pi$ ) in line 2 of Algorithm 1). If this is the case, we can check the consistency of each positive test case with the knowledge base (TESTC( $C \cup B \cup \overline{T}_\nu, T_\pi$ ) in line 4 of Algorithm 1) where TESTC returns the set of test cases in  $T_\pi$  inconsistent with

$C \cup B \cup \overline{T}_\nu$ . If some of the positive test cases induce an inconsistency ( $T'_\pi \neq \emptyset$  in line 5 of Algorithm 1), MSSDIRECT (Algorithm 2) is activated to identify those constraints part of  $C_{KB}$  which are responsible for the faulty behavior.

MSSDIRECT determines a maximum satisfiable subset  $\Omega \subseteq C_{KB}$  (MSS) [Marques-Silva *et al.*, 2015] which cannot be further extended with other constraints of  $C_{KB}$  without violating at least one of the test cases in  $T_\pi$ . A knowledge base diagnosis  $\Delta$  (see Definition 3) is the complement of an MSS  $\Omega$ , i.e.,  $C_{KB} - \Omega = \Delta$  and  $C_{KB} - \Delta = \Omega$  [Marques-Silva *et al.*, 2015]. MSSDIRECT (Algorithm 2) is activated with the diagnosis candidates  $C$  (constraints of  $C_{KB}$  considered as potentially faulty), the background knowledge  $B$  (consisting of  $C_{KB}$  constraints assumed to be correct and  $\overline{T}_\nu$ ), and the set of positive test cases  $T_\pi$ . The algorithm determines an MSS  $\Omega$ , the corresponding minimal diagnosis is  $\Delta = C - \Omega$  (see the Algorithm 1 result in line 6).

MSSDIRECT is based on the invariant that the background knowledge  $B$  is only extended with elements of  $C$  if  $\forall t_\pi \in T_\pi : C \cup B \cup \{t_\pi\}$  is consistent. If some positive test cases still induce an inconsistency and  $|C| = 1$  ( $\lambda = 1$ ), this means that  $C$  includes a diagnosis element (which can be removed). The parameter  $\lambda$  in Algorithm 2 can be used to control the minimality of diagnoses returned by KBDIAG: if  $\lambda = 1$ , MSSDIRECT returns a maximum satisfiable subset resulting in a subset-minimal diagnosis (the complement) returned by KBDIAG (see Definition 3). Otherwise ( $\lambda > 1$ ), there is a loss of diagnosis minimality (MSS maximality) which can, however, result in less consistency checks by MSSDIRECT (see also the analysis in Section 5). The mentioned minimality loss can be explained by the fact that if  $C \cup B \cup \{t_\pi\}$  is inconsistent for at least one  $t_\pi$  and  $|C| > 1$  it cannot be guaranteed that all elements in  $C$  are diagnosis-relevant.

The *background knowledge*  $B$  consists of constraints assumed to be correct, i.e.,  $B$  entails those constraints which should not be regarded as diagnosis candidates. Furthermore,  $B$  includes the *negative test cases*  $t_\nu \in T_\nu$  in *negated form* (which are assumed to be consistent with  $C \cup B$ ). Before starting MSSDIRECT, all  $t_\pi$  in  $T_\pi$  have to be checked for consistency with  $C \cup B \cup \overline{T}_\nu$  (line 4 of Algorithm 1).

If at least one positive test case induces an inconsistency in  $C \cup B \cup \overline{T}_\nu$ , MSSDIRECT is activated (line 6 in Algorithm 1). Only those test cases in  $T_\pi$  are forwarded to MSSDIRECT

---

**Algorithm 1** KBDIAG( $C = \{c_1..c_n\}, B, T_\nu, T_\pi, \lambda) : \Delta$ 

---

```
1:  $\overline{T}_\nu \leftarrow \bigwedge_{t_\nu \in T_\nu} \{\neg t_\nu\}$ 
2:  $T'_\pi \leftarrow \text{TESTC}(\overline{T}_\nu, T_\pi)$ 
3: if  $T'_\pi = \emptyset$  then
4:    $T'_\pi \leftarrow \text{TESTC}(C \cup B \cup \overline{T}_\nu, T_\pi)$ 
5:   if  $(T'_\pi \neq \emptyset)$  then
6:      $\text{return}(C - \text{MSSDIRECT}(\emptyset, C, B \cup \overline{T}_\nu, T'_\pi, \lambda))$ 
7:   else
8:      $\text{write}(\text{'all test cases satisfied'})$ ;  $\text{return}(\emptyset)$ 
9:   end if
10: else
11:    $\text{write}(\text{'inconsistent test cases'})$ ;  $\text{return}(\emptyset)$ 
12: end if
```

---

---

**Algorithm 2** MSSDIRECT( $\phi, C = \{c_1..c_n\}, B, T_\pi, \lambda) : \Omega$ 

---

```
1:  $T'_\pi \leftarrow T_\pi$ 
2: if  $\phi \neq \emptyset$  then
3:    $T'_\pi \leftarrow \text{TESTC}(C \cup B, T_\pi)$ 
4:   if  $T'_\pi = \emptyset$  then
5:      $\text{return}(C)$ 
6:   end if
7: end if
8: if  $|C| \leq \lambda$  then
9:    $\text{return}(\emptyset)$ 
10: end if
11:  $\alpha = \lfloor \frac{n}{2} \rfloor$ 
12:  $C_1 \leftarrow c_1..c_\alpha$ ;  $C_2 \leftarrow c_{\alpha+1}..c_n$ 
13:  $\Omega_2 \leftarrow \text{MSSDIRECT}(C_1, C_1, B, T'_\pi, \lambda)$ 
14:  $\Omega_1 \leftarrow \text{MSSDIRECT}(C_1 - \Omega_2, C_2, B \cup \Omega_2, T'_\pi, \lambda)$ 
15:  $\text{return}(\Omega_1 \cup \Omega_2)$ 
```

---

which are inconsistent with  $C \cup B \cup \overline{T}_\nu$ . In our setting, the original set of positive test cases  $T_\pi = \{t_{\pi 1}..t_{\pi 3}\}$  is reduced to  $\{t_{\pi 1}, t_{\pi 3}\}$  since  $t_{\pi 2}$  does not induce an inconsistency in  $C \cup B \cup \overline{T}_\nu$ , i.e., there exists a solution for  $C \cup B \cup \overline{T}_\nu \cup \{t_{\pi 2}\}$ .

MSSDIRECT determines a maximum satisfiable subset MSS ( $\Omega$ ) where  $C \subseteq C_{KB}$  (the consideration set),  $B = C_{KB} - C \cup \overline{T}_\nu$  (the background knowledge), and  $\overline{T}_\nu$  represents a conjunction of negated negative test cases. MSSDIRECT is activated with test cases  $T_\pi$  that are inconsistent with  $C \cup B \cup \overline{T}_\nu$ .  $C \subseteq C_{KB}$  can be used to focus diagnosis search on specific parts of a configuration knowledge base. If  $C_{KB}$  should be diagnosed as a whole,  $C = C_{KB}$  and  $B = \overline{T}_\nu$ .

TESTC (line 3 in Algorithm 2) checks whether the constraints in  $C \cup B$  are consistent with the test cases in  $T_\pi$ . Obviously, test cases have to be checked individually, i.e., each activation of TESTC results in  $|T_\pi|$  constraint solver activations (consistency checks). Only test cases inducing an inconsistency with  $C \cup B$  are stored (returned) in  $T'_\pi$  (the remaining inconsistent positive test cases). A consistency check is activated if  $\phi \neq \emptyset$  ( $\phi = \emptyset$  indicates that  $C$  has already been checked for consistency with  $B$  - see line 2 in Algorithm 2).

**MSSDIRECT Execution.** An execution trace of MSSDIRECT is shown in Figure 2. MSSDIRECT follows a *divide & conquer* approach. In each incarnation, it is analyzed which positive test cases remain inconsistent with  $C \cup B$ . If just one

constraint  $c_i$  remains in the consideration set  $C$  ( $|C| = 1$  if we assume  $\lambda = 1$ ) and there still exists at least one test case  $t_{\pi j}$  with  $\text{inconsistent}(\{t_{\pi j}\} \cup C \cup B)$ , then  $c_i$  is considered a part of a diagnosis and thus omitted in the determination of the maximum satisfiable subset  $\Omega$  (line 9 in Algorithm 2). If  $C \cup B$  is consistent with all test cases in  $T_\pi$ ,  $C$  is returned since no diagnosis elements can be found in  $C$ , i.e.,  $C$  is part of the MSS (line 5 in Algorithm 2).

In our example, MSSDIRECT returns a maximum satisfiable subset ( $\Omega = \{c_1, c_3, c_5\}$ ) (see Figure 2). To determine a corresponding minimal diagnosis  $\Delta$ , we have to calculate the MSS-complement of  $\Omega$ , i.e.,  $C - \Omega$  which is  $\{c_2, c_4\}$ .

## 5 Analysis of MSSDIRECT

We will now provide a theoretical analysis of MSSDIRECT and compare the performance of MSSDIRECT with hitting set based knowledge base diagnosis approaches [Reiter, 1987; Bakker *et al.*, 1993; Felfernig *et al.*, 2004]. Our motivation for choosing hitting set based diagnosis as baseline is that no other related test-driven knowledge base testing debugging approaches exist that could be used for evaluation purposes.

**MSSDIRECT Diagnosis Complexity.** The *worst-case* complexity of MSSDIRECT in terms of the number of needed consistency checks for determining one maximum satisfiable subset  $\Omega$  and the corresponding subset-minimal diagnosis  $\Delta = C - \Omega$  is  $\lceil (2d \times \log_2(\frac{n}{d})/\lambda) \rceil + 2d/\lambda$  where  $d$  represents the *set size of the minimal diagnosis*,  $n$  is the *number of constraints* in  $C$ , and  $2d$  represents the branching factor and the number of leaf-node consistency checks. If  $\lambda = 1$ , this factor can be reduced to  $2d \times \log_2(\frac{n}{d}) + 2d$ .

In this context, we interpret the checking of all positive test cases of  $T_\pi$  for consistency with  $C \cup B \cup \overline{T}_\nu$  as *one elementary consistency check*. The corresponding *best-case* complexity is  $\lceil (\log_2(\frac{n}{d})/\lambda) \rceil + 2d/\lambda$ . In the worst case, each diagnosis element is located in a different path of the search tree. The factor  $\log_2(\frac{n}{d})/\lambda$  represents the depth of a path of the MSSDIRECT search tree. In the best case, all constraints part of a diagnosis are included in a single path of the search tree. With an increasing size of  $\lambda$ , the number of needed consistency checks decreases (with an increasing  $\lambda$ -size, MSSDIRECT recursion depth is reduced, since the  $|C| \leq \lambda$  is fulfilled earlier) but at the same time diagnosis minimality cannot be guaranteed ( $C$  could include diagnosis-irrelevant constraints).

**HSDAG Diagnosis Complexity.** The *worst-case* complexity of QUICKXPLAIN – the conflict detection approach used in our baseline HSDAG algorithm [Felfernig *et al.*, 2004] – in terms of the number of consistency checks needed for determining one minimal conflict set is  $2c \times \log_2(\frac{n}{c}) + 2c$  where  $c$  is the size of a minimal conflict set and  $n$  is the number of constraints in  $C$  [Junker, 2004]. The *best-case* complexity of QUICKXPLAIN in terms of the number of needed consistency checks is  $\log_2(\frac{n}{c}) + 2c$ . The number of QUICKXPLAIN consistency checks per conflict set and the number of consistency checks per diagnosis determined by MSSDIRECT fall into the same complexity class (differing in the parameters  $d$  and  $c$ ). Let us assume that  $\beta$  represents the number of conflict sets in  $C$  and  $\mu$  the num-

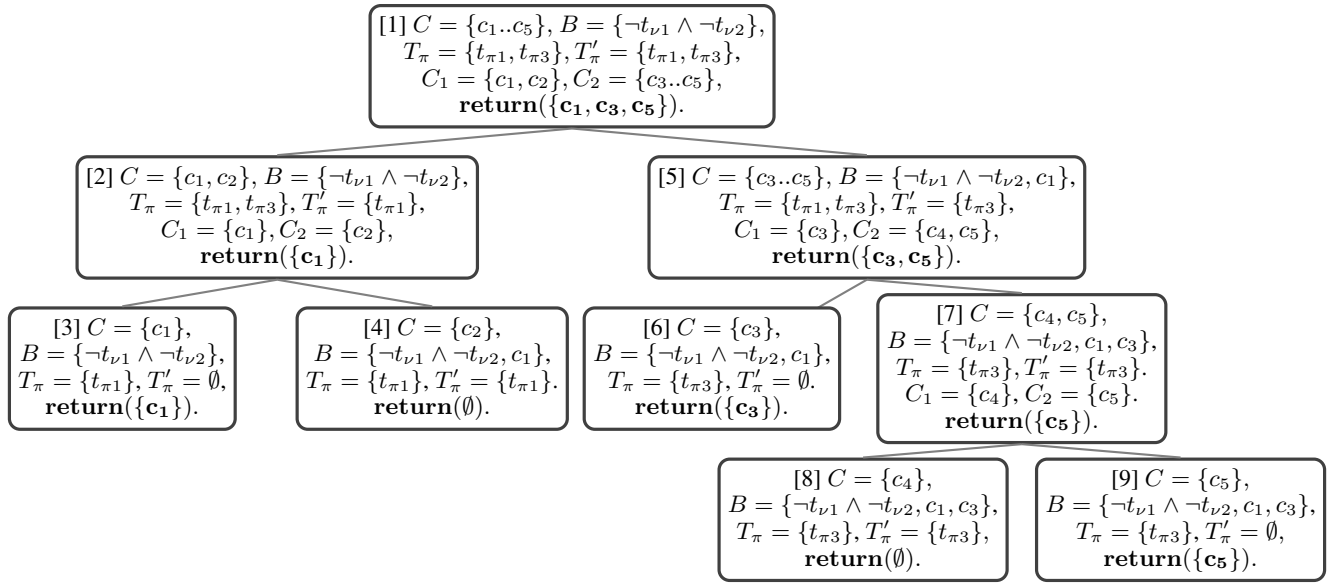


Figure 2: MSSDIRECT execution trace for  $C = \{c_1..c_5\}$ ,  $B = \{\neg t_{\nu 1} \wedge \neg t_{\nu 2}\}$ , and  $T_{\pi} = \{t_{\pi 1}, t_{\pi 3}\}$  assuming  $\lambda = 1$ . MSSDIRECT determines a maximum satisfiable subset ( $\Omega = \{c_1, c_3, c_5\}$ ), the corresponding diagnosis is  $\Delta = \{c_2, c_4\}$  (the  $\Omega$ -complement). Finding the leading  $n$  diagnoses (or all diagnoses) requires the creation of a HSDAG where individual nodes represent KBDIAG results.

| method | best case                                                  | worst case                                                           |
|--------|------------------------------------------------------------|----------------------------------------------------------------------|
| MSD    | $\lceil (\log_2(\frac{n}{d})/\lambda) + 2d/\lambda \rceil$ | $\lceil (2d \times \log_2(\frac{n}{d})/\lambda) + 2d/\lambda \rceil$ |
| HSD    | $d \times (\log_2(\frac{n}{c}) + 2c)$                      | $d \times (2c \times \log_2(\frac{n}{c}) + 2c)$                      |

Table 2: **Complexity** of MSSDIRECT (MSD) vs. complexity of HSDAG+QUICKXPLAIN (HSD) in terms of the number of needed consistency checks for *one diagnosis*. We interpret the check  $T'_{\pi} \leftarrow \text{TESTC}(C \cup B, T_{\pi})$  (line 3 of Algorithm 2) as *one* consistency check.

ber of corresponding minimal diagnoses. Then,  $\mu$  MSSDIRECT activations are needed for determining all diagnoses. In the context of a HSDAG-based approach [Wotawa, 2001; Felfernig *et al.*, 2004], we need  $\beta$  activations of QUICKXPLAIN for determining all minimal diagnoses in  $C$  (assuming a HSDAG implementation with conflict set reuse and path pruning).

**MSSDIRECT Runtime Performance.** We have evaluated the performance of MSSDIRECT (MSD with  $\lambda = 1$  and  $\lambda = 2$ ) compared to HSDAG+QUICKXPLAIN (HSD)<sup>2</sup> on the basis of real-world configuration knowledge bases selected from the S.P.L.O.T. configuration knowledge base repository [Mendonca *et al.*, 2009]. For each knowledge base, we have generated test suites (test case sizes with increasing cardinality) with a 20% share of inconsistency-inducing test cases where each test case is associated with one of the following *five patterns*: a selected variable value  $v = v_a$  (1) *should be included in a least one configuration* ( $t_{\pi} : v = v_a$ ), (2) *should not be part of each configuration* ( $t_{\pi} : v \neq v_a$ ), (3) *should be part of each configuration* ( $t_{\nu} : v \neq v_a$ ), and (4) *should not be part of any configuration* ( $t_{\nu} : v = v_a$ ). Furthermore, (5) we include *partial config-*

urations either as positive or negative test cases (referring to 1–5 randomly selected variables). For our evaluations, we assumed  $C = C_{KB}$ , i.e., all constraints in  $C_{KB}$  have been interpreted as diagnosis candidates.

All experiments have been conducted with an Intel Core i7 (6 cores) of 2.60GHz computer with 16-GB RAM. For evaluation purposes, we used the CHOCO solver.<sup>3</sup> Each  $|T_{\pi}| \times |C|$  entry in Table 3 represents *number of needed constraint solver calls (consistency checks) / average runtime in msec* for both, HSDAG+QUICKXPLAIN (HSD) and MSSDIRECT (MSD) with a repetition rate of 3 per setting.

The experimental results depicted in Tables 3 and 4 show that MSSDIRECT outperforms existing test case driven knowledge base diagnosis approaches which combine conflict detection [Junker, 2004] and conflict resolution based on hitting set directed acyclic graphs [Reiter, 1987; Bakker *et al.*, 1993; Felfernig *et al.*, 2004]. This holds for different numbers of returned (leading) diagnoses and also when determining all minimal diagnoses. Consequently, MSSDIRECT clearly outperforms HSDAG-based diagnosis approaches used for supporting the testing of configuration knowledge bases. In hitting set based approaches [Reiter, 1987; Bakker *et al.*, 1993; Felfernig *et al.*, 2004], the complete set of minimal diagnoses (or the leading  $n$  diagnoses) is determined on the basis of conflict resolution. When applying MSSDIRECT, these diagnoses are also determined using a HSDAG. However, individual nodes of the search tree are represented by diagnoses determined by MSSDIRECT.

To show the impact of the parameter  $\lambda$ , we have compared the runtimes of  $\lambda = 1$  and  $\lambda = 2$  in our evaluation settings. The corresponding results in the context of determining *one minimal diagnosis* are depicted in Table 3 indicating further

<sup>2</sup>The source code will be available on *GitHub*.

<sup>3</sup>choco-solver.org

| $ T_\pi $ | DELL ( $ C  = 121$ ) |                     |               | Ubuntu ( $ C  = 261$ ) |                    |               | Win8 ( $ C  = 405$ )  |                    |               |
|-----------|----------------------|---------------------|---------------|------------------------|--------------------|---------------|-----------------------|--------------------|---------------|
|           | HSD                  | MSD                 |               | HSD                    | MSD                |               | HSD                   | MSD                |               |
|           |                      | $\lambda = 1$       | $\lambda = 2$ |                        | $\lambda = 1$      | $\lambda = 2$ |                       | $\lambda = 1$      | $\lambda = 2$ |
| 5         | 95 / 20.8            | 20 / <b>2.2</b>     | 17 / 1.5      | 13 / 3.3               | 11 / <b>3.1</b>    | 10 / 3.0      | 73 / 71.3             | 11 / <b>8.0</b>    | 11 / 7.2      |
| 10        | 165 / 59.4           | 33 / <b>2.5</b>     | 29 / 1.9      | 73 / 41.2              | 22 / <b>7.2</b>    | 20 / 7.1      | 265 / 206             | 25 / <b>17.2</b>   | 23 / 14.3     |
| 25        | 2,401 / 700          | 82 / <b>5.0</b>     | 69 / 4.0      | 1,762 / 376            | 53 / <b>16.7</b>   | 48 / 15.0     | 2,560 / 2,450         | 64 / <b>43.6</b>   | 56 / 36.1     |
| 50        | 4,706 / 2,480        | 163 / <b>5.8</b>    | 137 / 5.8     | 25,502 / 9,734         | 97 / <b>31.3</b>   | 89 / 26.1     | 16,224 / 14,956       | 127 / <b>79.5</b>  | 111 / 60.0    |
| 100       | 7,572 / 5,016        | 344 / <b>11.1</b>   | 290 / 11.0    | 30,422 / 11,710        | 188 / <b>56.3</b>  | 171 / 48.5    | 108,351 / 183,700     | 277 / <b>179</b>   | 244 / 157     |
| 250       | 11,956 / 10,108      | 834 / <b>28.9</b>   | 694 / 23.6    | 698,292 / 762,142      | 549 / <b>169</b>   | 468 / 134     | 239,038 / 564,530     | 730 / <b>520</b>   | 649 / 431     |
| 500       | 14,331 / 14,141      | 1,668 / <b>55.3</b> | 1,389 / 43.7  | 1,284,377 / 1,455,601  | 1,258 / <b>395</b> | 1,121 / 333   | 2,102,863 / 5,726,443 | 1,232 / <b>869</b> | 1,100 / 706   |

Table 3: **Number of solver calls / average run-time performance** (in *msec* with rounded-up runtimes for values  $\geq 100$ ) of HSDAG+QUICKXPLAIN (HSD) versus MSSDIRECT (MSD with  $\lambda = 1$  and  $\lambda = 2$ ) needed for determining *one minimal diagnosis*. The evaluation has been conducted with three real-world configuration knowledge bases from S.P.L.O.T (*DELL*=DELL computer configuration, *Ubuntu*=LINUX UBUNTU operating system configuration, *Win8*=WINDOWS8 configuration).

| #leading<br>diagnoses ( $n$ ) | DELL ( $ C  = 121$ ) |               |               | Ubuntu ( $ C  = 261$ ) |               |               | Win8 ( $ C  = 405$ ) |               |               |
|-------------------------------|----------------------|---------------|---------------|------------------------|---------------|---------------|----------------------|---------------|---------------|
|                               | $ \Delta  = 40$      |               |               | $ \Delta  = 259$       |               |               | $ \Delta  = 561$     |               |               |
|                               | HSD                  | MSD           |               | HSD                    | MSD           |               | HSD                  | MSD           |               |
|                               |                      | $\lambda = 1$ | $\lambda = 2$ |                        | $\lambda = 1$ | $\lambda = 2$ |                      | $\lambda = 1$ | $\lambda = 2$ |
| 1                             | 699.5                | <b>5.0</b>    | 4.0           | 376.0                  | <b>16.7</b>   | 15.0          | 2,450.0              | <b>43.6</b>   | 36.1          |
| 2                             | 708.0                | <b>12.8</b>   | 10.4          | 470.8                  | <b>48.8</b>   | 39.1          | 2,512.1              | <b>88.8</b>   | 62.3          |
| 5                             | 810.3                | <b>36.7</b>   | 33.3          | 754.6                  | <b>119.0</b>  | 108.4         | 3,473.7              | <b>231.7</b>  | 157.5         |
| 10                            | 852.4                | <b>68.1</b>   | 68.9          | 1,232.6                | <b>235.3</b>  | 210.1         | 5,230.7              | <b>438.5</b>  | 332.3         |

Table 4: **Runtime performance** (in *msec*) of HSDAG+QUICKXPLAIN (HSD) versus MSSDIRECT (MSD with  $\lambda = 1$  and  $\lambda = 2$ ) for identifying the  $n$  leading diagnoses in three real-world configuration knowledge bases from S.P.L.O.T. In this context, we assume  $|T| = 25$  and a 20% share of violated test cases.  $|\Delta|$  denotes the total number of all minimal diagnoses (*DELL*=DELL computer configuration, *Ubuntu*=LINUX UBUNTU operating system configuration, *Win8*=WINDOWS8 configuration).

performance improvements at the cost of potentially non-minimal diagnoses. This way of indicating the acceptance of non-minimal diagnoses ( $\lambda = 1$  indicates that only minimal diagnoses are accepted) can also make sense in scenarios where highly efficient diagnosis determination is preferred over diagnosis minimality [Nica *et al.*, 2013]. Note that an increased  $\lambda$  could also deteriorate diagnosis performance with an increasing number of leading diagnoses (unnecessary consistency checks triggered by diagnosis-irrelevant constraints).

## 6 Conclusions

In this paper, we have introduced a test case driven direct diagnosis approach to the identification of faulty constraints in knowledge bases (using configuration as an example application domain) where conflicts are induced by test cases specifying the intended behavior of a knowledge base. In this context, a diagnosis is represented by a subset-minimal set of constraints. This approach can support knowledge engineers in identifying the sources of inconsistencies more efficiently, which can be extremely relevant for the development of large and complex knowledge bases. Our diagnosis approach is not restricted to constraint-based representations, i.e., it can as well be applied to other representations such as answer set programming, SAT solving, and description logics.

Open issues for future work are: (1) in the line of query selection strategies for diagnosis discrimination, we will develop concepts for estimating correctness probabilities of constraints to take into account aspects such as outdated knowledge thus helping to further improve the constraint ranking used as input for direct diagnosis, (2) we will analyze

the user acceptance of non-minimal diagnoses (in settings with  $\lambda > 1$ ), (3) we will analyze in which way constraint learning (e.g., finding  $\bar{T}_\nu$  generalizations) can further improve the quality of diagnosis feedback for knowledge engineers, and (4) we will analyze possibilities of direct CSP/SAT encodings for test-driven knowledge base diagnosis.

## References

- [Bachant and McDermott, 1984] J. Bachant and J. McDermott. R1 Revisited: Four Years in the Trenches. *AI Magazine*, 3(3):21–32, 1984.
- [Bakker *et al.*, 1993] R. Bakker, F. Dikker, F. Tempelman, and P. Wogmim. Diagnosing and solving over-determined constraint satisfaction problems. In *IJCAI'93*, pages 276–281. Morgan Kaufmann, 1993.
- [C  be and Wassermann, 2015] R. C  be and R. Wassermann. Ontology repair through partial meet contraction. In *Intl. Conference on Defeasible and Ampliative Reasoning*, volume 1423 of *DARE'15*, page 9–15, Aachen, Germany, 2015. CEUR-WS.org.
- [DeKleer *et al.*, 1992] J. DeKleer, A. Mackworth, and R. Reiter. Characterizing Diagnoses and Systems. *Artificial Intelligence*, 56(2–3):197–222, 1992.
- [Erdem *et al.*, 2016] E. Erdem, M. Gelfond, and N. Leone. Applications of answer set programming. *AI Magazine*, 37(3):53–68, 2016.
- [Falkner and Schreiner, 2014] A. Falkner and H. Schreiner. Configuration and Reconfiguration in Industry. In

- 577 A. Felfernig, L. Hotz, C. Bagley, and J. Tihihonen, editors,  
578 *Knowledge-based Configuration – From Research to Busi-*  
579 *ness Cases*, pages 251–264. Morgan Kaufmann, 2014.
- 580 [Feldman *et al.*, 2010] A. Feldman, G. Provan, J. de Kleer,  
581 S. Robert, and A. van Gemund. Solving model-based di-  
582 agnosis problems with Max-SAT solvers and vice versa.  
583 In *21st International Workshop on the Principles of Diag-*  
584 *nosis*, pages 185–192, Portland, Oregon, USA, 2010.
- 585 [Felfernig *et al.*, 2004] A. Felfernig, G. Friedrich, D. Jan-  
586 nach, and M. Stumptner. Consistency-based diagnosis  
587 of configuration knowledge bases. *Artificial Intelligence*,  
588 152(2):213 – 234, 2004.
- 589 [Felfernig *et al.*, 2012] A. Felfernig, M. Schubert, and C. Ze-  
590 hentner. An efficient diagnosis algorithm for inconsistent  
591 constraint sets. *AIEDAM*, 26(1):53–62, 2012.
- 592 [Fleischanderl *et al.*, 1998] G. Fleischanderl, G. Friedrich,  
593 A. Haselboeck, H. Schreiner, and M. Stumptner. Configur-  
594 ing large systems using generative constraint satisfaction.  
595 *IEEE Intelligent Systems*, 13(4):59–68, 1998.
- 596 [Fleischanderl, 2002] G. Fleischanderl. Suggestions  
597 from the software engineering practice for applying  
598 consistency-based diagnosis to configuration knowledge  
599 bases. In *13th Intl. Workshop on Principles of Diagnosis*  
600 *(DX-02)*, pages 33–35, Semmering, Austria, 2002.
- 601 [Greiner *et al.*, 1992] R. Greiner, B. Smith, and R. Wilker-  
602 son. *A correction to the algorithm in Reiter’s theory of di-*  
603 *agnosis*, pages 49–53. Morgan Kaufmann Publishers Inc.,  
604 San Francisco, CA, USA, 1992.
- 605 [Gupta *et al.*, 2021] S. Gupta, B. Genc, and B. O’Sullivan.  
606 Explanation in Constraint Satisfaction: A Survey. In *13th*  
607 *International Joint Conference on Artificial Intelligence*,  
608 *IJCAI-21*, pages 4400–4407, 2021.
- 609 [Junker, 2004] U. Junker. QUICKXPLAIN: preferred expla-  
610 nations and relaxations for over-constrained problems. In  
611 *AAAI 2004*, pages 167–172, 2004.
- 612 [Junker, 2006] U. Junker. Configuration. In F. Rossi, P. van  
613 Beek, and T. Walsh, editors, *Handbook of Constraint Pro-*  
614 *gramming*, pages 837–873. Elsevier, 2006.
- 615 [Marques-Silva *et al.*, 2015] J. Marques-Silva, M. Janota,  
616 A. Ignatiev, and A. Morgado. Efficient model based di-  
617 agnosis with maximum satisfiability. In *IJCAI’15*, pages  
618 1966–1972, Buenos Aires, Argentina, 2015. AAAI.
- 619 [McGuinness, 2007] D. McGuinness. Configuration. In  
620 F. Baader, D. Calvanese, D. McGuinness, D. Nardi, and  
621 P. Patel-Schneider, editors, *The Description Logic Hand-*  
622 *book: Theory, Implementation and Applications*, pages  
623 417–435. Cambridge University Press, 2007.
- 624 [Mendonca *et al.*, 2009] M. Mendonca, M. Branco, and  
625 D. Cowan. S.P.L.O.T.: Software Product Lines Online  
626 Tools. In *24th ACM SIGPLAN Conference Companion*  
627 *on Object Oriented Programming Systems Languages and*  
628 *Applications*, OOPSLA ’09, pages 761–762, New York,  
629 NY, USA, 2009. ACM.
- 630 [Metodi *et al.*, 2012] A. Metodi, R. Stern, M. Kalech, and  
631 M. Codish. Compiling model-based diagnosis to boolean  
satisfaction. In *AAAI 2012*, pages 793–799, Toronto, On-  
tario, Canada, 2012. AAAI.
- [Metodi *et al.*, 2014] A. Metodi, R. Stern, M. Kalech, and  
M. Codish. A Novel SAT-Based Approach to Model Based  
Diagnosis. *Journal of AI Research*, 51:377–411, 2014.
- [Nica and Wotawa, 2012] I. Nica and F. Wotawa. ConDiag –  
Computing Minimal Diagnoses Using a Constraint Solver.  
In *International Workshop on Principles of Diagnosis*  
*(DX’2012)*, pages 185–192, 2012.
- [Nica *et al.*, 2013] I. Nica, I. Pill, T. Quaritsch, and  
F. Wotawa. The Route to Success – A Performance Com-  
parison of Diagnosis Algorithms. In *IJCAI’13*, pages  
1039–1045, 2013.
- [Reiter, 1987] R. Reiter. A theory of diagnosis from first  
principles. *Artificial Intelligence*, 32(1):57–95, 1987.
- [Rodler, 2022] P. Rodler. A formal proof and simple ex-  
planation of the QuickXplain algorithm. *Artificial Intel-*  
*ligence Review*, 55(8):6185–6206, 2022.
- [Rossi *et al.*, 2006] F. Rossi, P. van Beek, and T. Walsh.  
*Handbook of Constraint Programming*. Elsevier, 2006.
- [Schlobach *et al.*, 2007] S. Schlobach, Z. Huang, R. Cornet,  
and F. Harmelen. Debugging Incoherent Terminologies. *J.*  
*Autom. Reason.*, 39(3):317–349, 2007.
- [Shchekotykhin *et al.*, 2012] K. Shchekotykhin,  
G. Friedrich, P. Fleiss, and P. Rodler. Interactive  
Ontology Debugging: Two Query Strategies for Effi-  
cient Fault Localization. *Journal of Web Semantics*,  
12–13:88–103, 2012.
- [Siddiqi and Huang, 2007] S. Siddiqi and J. Huang. Hierar-  
chical diagnosis of multiple faults. In *IJCAI 2007*, pages  
581–586, San Francisco, CA, USA, 2007. Morgan Kauf-  
mann Publishers Inc.
- [Sinz *et al.*, 2003] C. Sinz, A. Kaiser, and W. Küchlin. For-  
mal Methods for the Validation of Automotive Product  
Configuration Data. *AIEDAM*, 17(1):75–97, 2003.
- [Stern *et al.*, 2012] R. Stern, M. Kalech, A. Feldman, and  
G. Provan. Exploring the Duality in Conflict-Directed  
Model-Based Diagnosis. In *AAAI’12*, pages 828–834.  
AAAI, 2012.
- [Stumptner, 1997] M. Stumptner. An Overview of  
Knowledge-based Configuration. *AI Communications*,  
10(2):111–125, 1997.
- [Wotawa, 2001] F. Wotawa. A Variant of Reiter’s Hitting-Set  
Algorithm. *Inf. Process. Lett.*, 79:45–51, 2001.